

os161_abc

Your portal to resources on OS161 at UBC.

[Home](#) [OS161 installation](#) [Understanding and Debugging](#) [OS161 Manual](#) [Contact Us](#)

Install OS 161 Tools in a Docker Container

Useful Links

Docker

Docker is a platform that allows developers to create, deploy, and run applications inside containers. Containers are lightweight, portable units that bundle an application and all its dependencies, ensuring that the application runs consistently across different environments. Before you begin, you need to make sure that you have Docker installed and running on your system.

[Build OS161](#)

[Debug OS161](#)

[Browse source code](#)

[How to use git](#)

Key Concepts

Host: This is the physical or virtual machine where Docker is installed. It provides the resources (like CPU, memory, storage) needed to run containers.

Guest: The guest is the container itself. It runs the application in an isolated environment, mimicking a full operating system but sharing the host's kernel. Each guest (container) thinks it's running on its own machine, but in reality, it's just a process on the host.

The main difference is that the host manages the resources and the environment, while the guest runs the application in a controlled, isolated space.

Prep your work directory for Docker

We will assume that the directory where you keep the clone the OS 161 repository is called: `/home/OS_BASE`.

```
cd /home/OS_BASE
```

If you have not already cloned your repository, this is a good time to do it.

Build the container

Next you will download and run the script to set up the Docker container with the required environment for running os161.

```
wget https://people.ece.ubc.ca/~os161/download/os161-cpen331-docker.tar.gz
gunzip os161-cpen331-docker.tar.gz
tar -xvf os161-cpen331-docker.tar
cd os161-cpen331-docker
docker build -t os161 .
```

Run the container

The following command will run the Docker container and will mount your home directory so that it is accessible from within the container. In the following command replace `/home/OS_BASE` with the name of the current directory.

```
docker run -dit --mount type=bind,src=/home/OS_BASE,target=/root/os161 --rm
```

The previous command printed the container ID, which you will use for the next command. You may also obtain the container ID anytime by running:

```
docker ps
```

Next you will get a shell to enter your container, using the container ID.

```
docker exec -it /bin/bash
```

Now you are inside the container! In other words, you are running inside the guest. If you want to open another shell in your container, repeat the above command. Next, execute the following command:

```
cd ~/os161
```

The os161 directory you just entered maps to `home/OS161_BASE` on the host. In that directory you will see the clone of your repo that you brought into `src` directory. If you have not yet cloned your repo, you can open a separate shell on the host and [clone it](#) into `home/OS161_BASE` under the name `src`.

There is one extra step to prepare for debugging OS161 if you are on MacOS (and possibly on other platforms which we haven't tested). From `~/os161`, execute:

```
mkdir root
mkdir root/.sockets
chmod 777 root/.sockets
```

You need to repeat this step every time you stop the container. If you simply exit the container by typing 'exit' in the shell and then re-enter it by opening another shell, you don't need to repeat this step. Some students reported that they have to run:

```
cd .sockets
ls
```

every time before opening the gdb connection and after closing. This is a bug in docker and we are not aware of a better work-around at the time of this writing. Then proceed building OS161 as [instructed on the website](#).